DAY 13 | HANDS-ON ACTIVITIES

Prompt Engineering: Problem Statements

Seven problems from warm-up to stretch. Every notebook cell runs offline; add an OPENAI_API_KEY to see real model behaviour. Offline results show how the code works, not real model quality.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Prompt Makeover

Level: Warm-up | **Concepts:** prompt anatomy

Take the prompt "write about our product" and rewrite it with role, task, context, format, examples and constraints for a SmartDesk 5G plan announcement. Compare the outputs (online) or the prompts (offline).

Sample run / expected result

```
Before: 4 words
After: role + task + 3 facts + format (3 bullets, under 60 words) + 1 example
```

Hints

- Specific beats long.
- Each added line should remove one guess.

Problem 2: Few-shot Classifier

Level: Easy | **Concepts:** zero-shot vs few-shot

Build a ticket classifier prompt (billing, network, account, other) with three examples. Run it on 8 tickets and measure accuracy against your own labels, zero-shot and few-shot.

Sample run / expected result

```
zero-shot accuracy 5/8
few-shot  accuracy 7/8
```

Hints

- Ask for exactly one word so you can compare easily.

Problem 3: Chain of Thought, Checked

Level: Medium | **Concepts:** CoT, verification

Ask for the annual cost of a 1,200 yen plan with 2 months free and 18% tax, step by step. Then check the number with Python and print whether they match.

Sample run / expected result

```
Model answer: 14160
Python check: 14160.0
Match: True
```

Hints

- Ask for the final line as Answer: <number> so you can parse it.

Problem 4: ReAct with a Safe Calculator

Level: Medium | **Concepts:** ReAct loop, tools

Extend the notebook's ReAct agent with a convert_currency tool (fixed placeholder rate) and answer "What is the GST on P-200 in Japanese yen?". The calculator must not use eval.

Sample run / expected result

```
Thought: need price -> lookup_price
Thought: need GST -> calculator
Thought: convert -> convert_currency
Final Answer: ...
```

Hints

- Limit the loop to 6 steps.
- Use the ast-based safe_calc from the notebook.

Problem 5: Prompt a Reasoning Model NEW (v4)

Level: Medium | **Concepts:** goal, constraints, success criteria

Rewrite a step-by-step prompt for planning a SmartDesk night-shift rota as a reasoning-model prompt: goal, constraints and success criteria only. Explain why you removed the steps.

Sample run / expected result

```
Goal: ...
Constraints: max 3 agents per shift; Japanese speaker on every shift
Success: JSON with shifts, one risk, one recommendation
```

Hints

- Fewer instructions about how; more about what good looks like.

Problem 6: Structured Ticket Labels NEW (v4)

Level: Challenging | **Concepts:** JSON Schema, Pydantic, templates

Define a Pydantic TicketLabel (category, urgency, summary). Send its JSON Schema with a versioned template, validate every reply, and fall back to category "other" with needs_review=True when validation fails.

Sample run / expected result

```
T-1 billing  high   valid
T-2 network  medium valid
T-3 other    low    fallback (needs_review)
```

Hints

- model_json_schema() gives you the schema.
- Catch ValidationError, do not crash.

Problem 7: Injection-Aware, Cache-Friendly Prompt NEW (v4)

Level: Stretch | **Concepts:** prompt injection, caching

Reorder a prompt so the stable parts come first. Add delimiters around the ticket text. Run five test tickets, including two injection attempts, and show that the labels stay inside the allowed list.

Sample run / expected result

```
stable prefix tokens: 180 of 230 (78% cacheable)
injection tickets: 2 of 2 labelled safely
```

Hints

- Count tokens of the prefix and the full prompt.
- Never let the output trigger an action alone.